2025-2026

Welcome to the 12th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please perform the Second Task (**B**) left from the last week Lab_Session_11_week_11	50	The second step will help you how to develop your own NN for different tasks. Build your deep understanding.
	Break	05	
B	In this task, we aim to implement the multi-layer Artificial Neural Network. The concepts here are the total reflection from the lecture of this week.	55	The activity '*Development of Neural Network (forward and backward propagation)*' start from Page 2 to Page 6 with detailed instructions and description of each snippet, need to be completed. Similarly, before starting the code, please read the instructions and the given description.

For One Hour Lab Session:

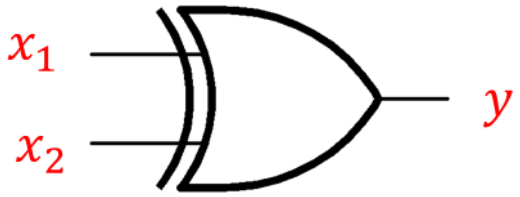
#	Task	Time (m)	Guidance
1	Please perform the Task **B** from above two hours lab session with the given instructions	50	Follow the related guide in Task **B**.

I recommend to download this PDF file so that you can view each line of code clearly.

Activity: Development of Neural Network (forward and backward propagation)

Important Note: Every student advised to carefully review each line and develop a clear understanding of the concepts. As we progress to more advanced and technical sessions in the coming weeks, it is essential to have a strong grasp of these foundational concepts. This will also support your preparation for the second assessment.

In the previous week, we implemented single perceptron/NN giving the prediction as **0** or **1**. This week, the neural network with multi-neurons is to be implemented to solve the **XOR** problem, a binary classification task where the output is 1 if exactly one of the inputs is 1, and 0 otherwise.
The **XOR** problem is significant because it is not linearly separable, meaning a single-layer model cannot solve it. Instead, we need a multi-layer neural network with at least one hidden layer and non-linear activation functions. This figure and table are given for understanding the XOR



Input1 x_1,	**Input2** x_2	**Output** y
0	0	0
0	1	1
1	0	1
1	1	0

The details of the layers as discussed in the lecture which will be applied in NN are:

Input Layer:
Takes the 2 input features (X= [[x_1, x_2]].
Hidden Layer:
Contains 4 neurons, which is sufficient for this small problem.
Uses a sigmoid activation function to introduce non-linearity.
Output Layer:
Contains 1 neuron to produce the final binary classification output.
Also uses the sigmoid activation to squash the output to a probability range [0,1].

Key Components of the Code

The code implements all essential elements of a neural network from scratch using NumPy, focusing on:
Data Preparation: The XOR dataset serves as both the training and testing data since the problem is small and well-defined.
Forward Pass: Propagates the input data through the network to compute predictions using matrix operations and activation functions.
Loss Calculation: Measures the difference between predicted outputs and true labels using Mean Squared Error (MSE).
Backward Pass: Computes the gradients of the loss with respect to weights and biases using the chain rule. Updates the parameters using gradient descent.

Training Loop: Repeats the forward and backward passes for multiple epochs (10,000 in this case) to minimize the loss.

Why these steps are taken?

We want to obtained and solve the XOR problem which I defined in the start.

Learning to Solve the XOR Problem: By minimizing the loss function over epochs, we aim to adjust the weights and biases so the network can correctly classify all four inputs of the XOR dataset.

Binary Classification Outputs:

After training, the model should output probabilities close to the true labels (0,1,1,0). We can use a threshold (e.g., 0.5) to convert these probabilities into binary classifications (0 or 1).

Start the implementation at this point.

Prepare data

This represents the XOR dataset, a classic problem where the output is 1 if exactly one of the inputs is 1, otherwise it is 0. Inputs have 2 features, and there are 4 examples.

XOR is a non-linear problem, so a neural network requires at least one hidden layer with non-linear activation functions to solve it.

```
[1]
import numpy as np
import matplotlib.pyplot as plt

# Sigmoid activation function which I explained in the lecture of this week.
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Initialize the dataset (X: inputs, y: outputs)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Input data (4 examples, 2 features). Once you read the description, from the table on Page 2.
y = np.array([[0], [1], [1], [0]])  # Target output (XOR problem) Once you read the description, from the table on Page 2.
```

Network Structure and Hyperparameters

The input layer has 2 neurons corresponding to the 2 features. The hidden layer has 4 neurons, which is an arbitrary choice. The output layer has 1 neuron since this is a binary classification problem. Learning rate (η) and number of epochs control the training process.

The number of hidden neurons can affect performance. Too few neurons may lead to underfitting, while too many may cause overfitting. Learning rate needs to be carefully tuned; a large value can cause divergence, while a small value can lead to slow convergence.

```
[2]
# Hyperparameters
input_neurons = X.shape[1]  # Number of input features (2)
hidden_neurons = 4          # Number of neurons in the hidden layer
output_neurons = 1          # Output layer (binary classification)
learning_rate = 0.1
epochs = 10000
```

Weight and Bias Initialization

Random initialization of weights ensures symmetry breaking, which is crucial for effective learning. Biases are initialized to zero. Using a fixed random seed ensures reproducibility of the results.

Initializing weights to very large values can lead to instability, while initializing to very small values can slow down training. Modern initialization techniques like Xavier or He initialization can improve convergence.

```
[3]  # Initialize weights and biases
     np.random.seed(42)  # For reproducibility
     W1 = np.random.uniform(-1, 1, (input_neurons, hidden_neurons))  # Weights for input to hidden layer
     b1 = np.zeros((1, hidden_neurons))                              # Biases for hidden layer
     W2 = np.random.uniform(-1, 1, (hidden_neurons, output_neurons)) # Weights for hidden to output layer
     b2 = np.zeros((1, output_neurons))                              # Biases for output layer
```

Forward pass

Z1 is the weighted sum of inputs plus bias for the hidden layer. A1 applies the sigmoid activation to introduce non-linearity. Z2 and A2 repeat the process for the output layer. The sigmoid function introduces non-linearity, enabling the model to learn complex relationships. However, sigmoid can cause vanishing gradient problems when used in deep networks.

Loss Calculation

Mean Squared Error (MSE) measures the difference between predicted outputs ($A2$) and true outputs (y). MSE is not ideal for classification tasks. For binary classification, binary cross-entropy is a better choice

Backpropagation

Gradients are computed for weights and biases in both layers using the chain rule. Gradients for the output layer propagate back to the hidden layer. Using the sigmoid derivative can cause issues in deep networks due to small gradients (vanishing gradient problem).

Weight updates

Parameters are updated using gradient descent. Gradient descent can be enhanced with optimizers like Adam, RMSProp, or Momentum for faster and more stable convergence.

```
[4]
# Store loss values for visualization
losses = []

# Training loop
for epoch in range(epochs):
    # Forward pass
    Z1 = np.dot(X, W1) + b1  # Input to hidden layer
    A1 = sigmoid(Z1)         # Activation of hidden layer
    Z2 = np.dot(A1, W2) + b2 # Input to output layer
    A2 = sigmoid(Z2)         # Activation of output layer (predicted output)

    # Compute loss (Mean Squared Error)
    loss = np.mean((y - A2) ** 2)
    losses.append(loss)

    # Backward pass
    dA2 = -(y - A2)                        # Derivative of loss w.r.t. output
    dZ2 = dA2 * sigmoid_derivative(A2)     # Derivative of output layer activation
    dW2 = np.dot(A1.T, dZ2)                # Gradient for W2
    db2 = np.sum(dZ2, axis=0, keepdims=True) # Gradient for b2

    dA1 = np.dot(dZ2, W2.T)                # Backpropagated error to hidden layer
    dZ1 = dA1 * sigmoid_derivative(A1)     # Derivative of hidden layer activation
    dW1 = np.dot(X.T, dZ1)                 # Gradient for W1
    db1 = np.sum(dZ1, axis=0, keepdims=True) # Gradient for b1

    # Update weights and biases
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

    # Print loss at intervals
    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

Here, you will find that the loss is printed every 1000 epochs to monitor the training process and the plot for loss over epochs for better understanding, is given. Then, the trained model is used to make predictions on the inputs. The output is a probability, and a threshold (e.g., 0.5) should be applied to classify as 0 or 1:

```
[6]
#Plot the loss curve
plt.figure(figsize=(10, 6))
plt.plot(losses, label="Loss over epochs")
plt.title("Training Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)
plt.legend()
plt.show()

# Testing the trained model
print("\nFinal predictions:")
A1 = sigmoid(np.dot(X, W1) + b1)
A2 = sigmoid(np.dot(A1, W2) + b2)

# Display predictions
for i in range(len(X)):
    print(f"Input: {X[i]}, Predicted: {A2[i][0]:.4f}, Actual: {y[i][0]}")

# Classify predictions (round to nearest integer)
predicted_classes = (A2 > 0.5).astype(int)
print("\nPredicted Classes:")
for i in range(len(X)):
    print(f"Input: {X[i]}, Predicted Class: {predicted_classes[i][0]}, Actual Class: {y[i][0]}")
```

After you finish this activity, you realized how the information is passed through each layer starting from the input to output layer. We covered single layer/neuron, multi-neuron/multi-layer, feedforward and backward propagation, forming feedforward neural network or ANN with examples. The understanding about the activation functions, errors, and loss have further made your ways towards advanced deep learning.